DSA Day-85/1-11-26

Leetcode-6 = ZigZag conversion. __/__/____

You are given: A string s. an int. numRows, representing the no. of rows.
You need to write the characters of the string in a zigzag pattern & then read the characters row by row.

example-1: S = "PAYPALISHIRING", numRows = 3.
→ first, arrange the characters in a zigzag pattern:-

P A N H
A P L S I I G
Y I R

now read row by row: • Row 1: PANH
• Row 2: APLSIIG
• Row 3: YIR.

final answer:- "PANHAPLSIIGYIR".

★ we are not rearranging the chars alphabetically or removing the string. We are simply changing their arrangement according to the zigzag pattern.

→ See the zigzag movement, lets see how the the chars will in 4 rows.

(x2)

P I N
A L S I G
Y A H R
P I

the movement follows 2 directions.
• diagonally upward: Row 3 → Row 2 → Row 1 → Row 0.
this movement keeps repeating until all characters are placed.

example 3 input: s = "A", numRows = 1.
output: "A".

Mathematical index pattern (approach)
instead of simulating the zigzag movement, we directly calculate the positions of characters belonging to each row.
observe the zig zag pattern carefully.
for: s = "PAYPALISHIRING", numRows = 3

pattern :→ P A H N
A P L S I I G
Y I R

notice, the 1st row contains characters at index:-
0, 4, 8, 12.
the last row contains: 2, 6, 10.
the middle row contains: 1, 3, 5, 7, 9, 11, 13.
this is a repeating mathematical pattern!!!
→ calculate the cycle length, the formula is,
cycleLen = 2 × numRows - 2.
why? → a complete zig zag cycle consists of:-
numRows characters moving downward.
numRows - 2 characters moving diagonally upwards.
like now, row 2 → next row diagonally.
rn4 = 2 - 1 = ——→ 1, next → row 0 not
counted as, the A starts the cycle again, it
will be counted twice, so, upto row 1.

therefore:

numRows * (numsRows - 2) = 2numsRows - 2.

for numRows = 3:

cycleLen = 2 (3) - 2 = 4.

So, every complete cycle spans 4 positions.

Q2 → understand the index pattern.

for each row, we calculate 2 possible char positions.

vertical position: j + row

diagonal " : j + cycleLen - row

here, j = the beginning of the current cycle.

row = the current row

cycleLen = the dist. bet^n consecutive cycles.

→ for the 1st & last row, there is no diagonal character, so we only use the vertical position.

where does j + row come from?

↳ lets take Row 2, ← from example (2).

during the 1st downward movement:-

Row 0 → index 0

Row 1 → index 1

Row 2 → index 2

Row 3 → index 3.

So, the index is simply, starting index + row no.

for the 1st cycle, starting index j = 0.

for row 2, j = 0, row = 2,

index = 0 + 2 = 2.

So, we get s[2], which is Y. That's why: - j + row.

Where does j + cycle len - row come from? __/__/____

→ we already know that one complete cycle has 6 chars.
downward = 4 chars. → from example 2.
diagonal = 2 additional chars.
total = 6.
So, the next cycle begins at index 6.
The diagonal movement: index 6 (I) R0
↑
index 5 (L) R1
↑
index 4 (A) R2
↑
index 3 (P) R3.

Let's calculate:-
for Row 2, the diagonal char is at index 4.
for Row 1, the " " " " " 5.

how do we get there?
we start from the next cycle's starting index (6)
& move backwards.
Row 2 : 6 - 2 = 4
Row 1 : 6 - 1 = 5.
that's it! the next cycle starts at j + cycle len.
∴ - j + cycle len - row.
we subtract the row no. bcuz the diagonal chars are positioned just before the next cycle's starting index.

class Solution { public:
string convert(string s, int numRows) {
if (numRows == 1 || numRows >= s.size()) return s; → all chars will be placed in the same row, reading row by row gives us the exact same thing.

s = "ABCDE"; numRows = 1; →

s = "ABC"; numRows = 5; →

3 chars, 5 rows.
Row 0: A
Row 1: B
Row 2: C
Row 3:
Row 4:

We don't have enough chars to move down & then diagonally up. Reading row by row shall give → "ABC".

string result = ""; → this stores the final zigzag-converted string.

int cycleLen = 2 * numRows - 2; → calculate cycle length. → this tells us the dist. bet^n the starting positions of consecutive zigzag cycles.

We process 1 row at a time, 1st 0, 1, 2 & so on, this ensures that chars are added in the result in the required row-by-row order.

for (int row = 0; row < numRows; row++) { traverse each cycle { for (int j = 0; j + row < s.size(); j += cycleLen) {

here, j starts at 0, we jump by cycleLen each time, j + row must remain a valid index.

for 3 rows, j takes 3 values:- 0, 1 & 2.

result += s[j + row]; this calculates the required position for the current row.

int diagonal = j + cycleLen - row; this calculates the " char position in the current cycle.

check whether a diagonal char exists.

```
if (row != 0 && row != numRows - 1 &&
        diagonal < s.size()) {
```

3 conditions:-

row != 0: the 1st row has no diagonal char.

row != numRows - 1: the last row has no " ".

diagonal < s.size(): the calc. index must be within the str.

If the cond^n are satisfied, we add the diagonal char.

```
result += s[diagonal]; } } }
return result; }
```

returns the result.

Hence, we processed every row in order & constructed the required string.

T.C = O(n)
S.C = O(n)

→ we visit each row
→ we calculate the char positions using mathematical choices.
→ every char is added exactly one
→ we don't perform unnecessary traversals of the original string.

dry run ⇒ S = "ABCDEF"
numRows = 3.

A	E	⇒	A(0)		E(4)
B	D	F	B(1)	D(3)	F(5)
C			C(2)		

cycleLen = 2 * numRows - 2; = 2 * 3 - 2; = 4;
So, every cycle starts after 4 indices.

find chars row by row,
Row 0, vertical formula: j + row
- j = 0 ⇒ 0 + 0 = 0 → A
- j = 4 → 4 + 0 = 4 → E. (AE)

Row 1,
j = 0 ⇒ vertical: 0 + 1 = 1 → B
diagonal: 0 + 4 - 1 = 3 → D
j = 4 → vertical: 4 + 1 = 5 → F (BDF)
diagonal: 4 + 4 - 1 = 7 → invalid, ignore.

Row 2; j = 0 ⇒ 0 + 2 = 2 → C. (C)
combine all rows, "AE BDF C"
without spaces, "AEBDFC"